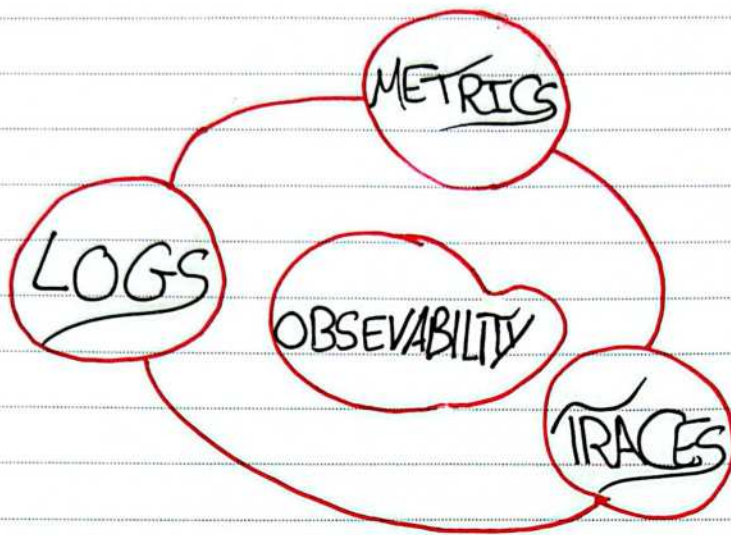




$Observability = Metrics + Logs + Traces$ #

أي هو الـ **Observability** ؟
 لو عندك Production فيه 500 Pods و 40 Microservices و Clusters و AWS والـ Client اتصل و بيشكك ان الموقع بيطيح ساعتها هنبقى عاجزين نعرف فين المشكلة ممكن **Network, DB, API, RAM, CPU** وانت للأسف لسا هتشت عارف هنا بيبدأ دور الـ **Observability**

الفرق بين الـ **Monitoring** و الـ **Observability**
 الـ **Monitoring** انت بتراقب حاجة انت عارفها زي الـ **Memory, Disk, Pod**

الـ **Observability** بتقدر تعرف سبب المشكلة حتى لو انت ما كنتش متوقعها
 يعني: مش بس بتعرف ان الموقع واقع انت كمان بتعرف ليه واقع

اكد هائل نفسي في الفرق بين الـ Observability VS Monitoring

معرفة الـ Monitoring و Observability Concepts مثل Tools التي تبني عبارة عن

الـ Monitoring عبارة عن Concept وبالنسبة لـ Tools

Prometheus

Nagios

Zabbix

Monitoring

→ Prometheus

→ Nagios

→ Zabbix

دول Tools يتعمل يعني

الـ Observability عبارة عن Concept وبالنسبة لـ Tools

Prometheus *

Grafana *

Tempo *



طيف العلاقة ما بين

Observability

Metrics

Logs

Traces

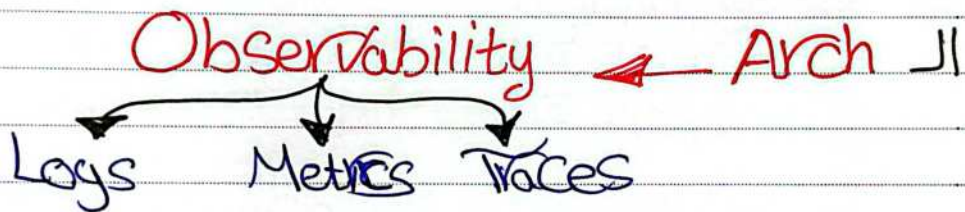
Monitoring

يعني الـ Monitoring بيتعمل على

ال Observability اتعملت ليه ؟

نماذج كان فيه **Monolithic Application**
كان كله عبارة عن **Service** واحدة
وكنه ساعتها هتلاقى ال **Error** بسهولة

لكن دلوقتي **Microservices**
وعنده **Services** كتير جدا
عشان كذا ظهر مفهوم ال **Observability**



يعني هو عبارة عن **Concept**
بيكون من 3 اعمدة

↓ **Metrics** ← دي عبارة عن ارقام
مثلا **CPU=70%** **Ram=8** **Disk=80%**
كلها ارقام

الشهر **Tools** ← **Prometheus** / **Grafana**
ال **Metrics** مش بتقولك سبب المشكلة
فيه هتقولك مثلا **CPU 95%** وانت هتعرفه
ان في مشكلة من غير ما تعرف السبب

Logs ← عبارة عن رسائل

Application Start
Connected to DB
User Login
Order Created

مثلا

كل دي عبارة عن Logs

2026-7-22

Error

Database Connection Faild

كدا عرفنا السبب

يعني **Metrics** قالت **CPU=95%**
Logs قالت **Database Timeout**

Database
هنا يعني دور ال **Traces**

ودي اصعب وامية فيهم

لكن أهم حاجة في ال **Microservices**

لو عندك **User** ← **Front** ← **API** ← **Auth**
↓ **orders** → **Payment** → **Inventory** → **DB**

كل ال **Request** ده عدى على **Services** كتير

لو ال **Payment** وقعت

هنا يعني ال **Trace**

ال **Trace** بي عمل **Tracking** لل **Request**
من أولها لآخرها

Order ← 20ms **Auth** ← 5ms **GW** ← 15ms **Front** ← **User**
↓ 40ms **Payment** ← 45sec **DB** ← 3ms

متلاحظ ان ال **Payment** هي اللى واطنة 4 ثوانى

يعني المشكلة هنا ودا اسمها **Distributed Tracing**

طيب في الفرق الحقيقي ؟

Matrics بتقولك في مشكلة

Logs انه اللي حصل

Traces المشكلة حصلت فين اثناء رحلة ال **Request**

يبقى لما الموقع يبقى بطيء

هتفتح **Grafana** وتشوف ال **Metrics**
هتلاقي ال **Latency** عاليه

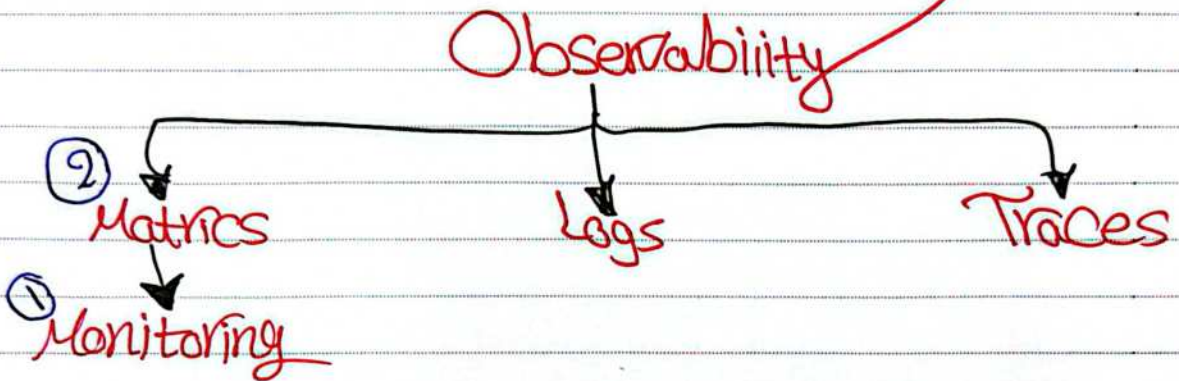
بعدها هتروح ال **Logs** هتلاقي **DB Time out**

بعدها هتفتح **Jaeger** ودي **Tool** خاصة بال **Traces**
وتهوف بعدها ال **Trace**
وتهتلاقي ال **Payment** فيها مشكلة
وبكده عرفت السبب الحقيقي

بعدها كل حاجة يروح ال **Grafana**
فتشوف كل حاجة في **Dashboard** واحدة



أول حاجة بتعرفك ان في مشكلة
هذه الـ **Monitoring** من خلال الـ **Metrics**
وبعدها تستخدم باقي عناصر الـ **Observability** عشان
تفهم السبب الحقيقي



يبقى الـ **Monitoring** بيتابع المؤشرات

الفرق بين الـ **Monitoring** و **Observability**

← **monitoring** بيقولنا لو النظام سليم أو لا
بناءً على مقاييس وتنبؤات من خلال
مراقبة الـ **Metrics**

← **Observability** يتجمع كل المقاييس
والسجلات وتعرف سبب المشكلة من لو
من متوقفتها

Monitoring = بيركز على اكتشاف وجود مشكلة (**Detection**)
Observability = بتشمل اكتشاف التشخيص، التعليل
(**Detection + Diagnosis + Root Cause Analysis**)
الـ **Monitoring** جزء من منظومة الـ **Observability**